

[illegible]

- lineBreaks: the number of line breaks found in the match. (Always zero if this rule has lineBreaks: false.) - line: the line number of the beginning of the match, starting from 1. - col: the column where the match begins, starting from 1. Value vs. Text The value is the same as the text, unless you provide a value transform.	<pre>const moo = require('moo') const lexer = moo.compile({ ws: /[\t]+/, string: {match: /"(?:\\["\\] ^\\n"\\ \\)*"/, value: s => s.slice(1, -1)}, }) lexer.reset('"test"') lexer.next() /* { value: 'test', text: "test", ... } */</pre>	Reset Calling reset() on your lexer will empty its internal buffer, and set the line, column, and offset counts back to their initial value. If you don't want this, you can save() the state, and later pass it as the second argument to reset() to explicitly control the internal state of the lexer.	<pre>lexer.reset('some line\n') let info = lexer.save() // -> { line: 10 } lexer.next() // -> { line: 10 } lexer.next() // -> { line: 11 } // ... lexer.reset('a different line\n', info) lexer.next() // -> { line: 10 }</pre> Keywords Moo makes it convenient to define literals.
Keywords can also have individual types. Keyword Types The keywords helper checks matches against the list of keywords, if any of them match, it uses the type 'keyword' instead of 'identifier' (for this example). now you'll lex class wrong (as an identifier). The keywords helper checks matches against the list of keywords, if any of them match, it uses the type 'keyword' instead of 'identifier' (for this example).	You need to do this to ensure the longest match principle applies, even in edge cases. Imagine trying to parse the input className with the following rules: <pre>keyword: ['class'], identifier: /[a-zA-Z]+/,</pre> You'll get <i>two</i> tokens — ['class', 'Name'] — which is <i>not</i> what you want! If you swap the order of the rules, you'll fix this example, but	<pre>moo.compile({ IDEN: {match: /[a-zA-Z]+/, type: moo.keywords({ KW: ['white', 'tf', 'else', 'moo', 'cows'], })}, SPACE: {match: /\s+/, lineBreaks: true}, })</pre>	<pre>moo.compile({ lparen: '(', rparen: ')', keyword: ['white', 'tf', 'else', 'moo', 'cows'], }) It'll automatically compile them into regular expressions, escaping them where necessary. Keywords should be written using the keywords transform.</pre>
<pre>let lexer = moo.compile({ name: {match: /[a-zA-Z]+/, type: moo.keywords({ 'kw-class': 'class', 'kw-def': 'def', 'kw-if': 'if', })}, // ... }) lexer.reset('def foo') lexer.next() // -> { type: 'kw- def', value: 'def' }</pre>	<pre>lexer.next() // space lexer.next() // -> { type: 'name', value: 'foo' }</pre> You can use Object.fromEntries to easily construct keyword objects: <pre>Object.fromEntries(['class', 'def', 'if'].map(k => ['kw-' + k, k]))</pre>	States Moo allows you to define multiple lexer states. Each state defines its own separate set of token rules. Your lexer will start off in the first state given to moo.states({}). Rules can be annotated with next, push, and pop, to change the current state after that token is matched. A "stack" of past states is kept, which is used by push and pop. next: 'bar' moves to the state named bar. (The stack is not changed.)	- push: 'bar' moves to the state named bar, and pushes the old state onto the stack. - pop: 1 removes one state from the top of the stack, and moves to that state. (Only 1 is supported.) Only rules from the current state can be matched. You need to copy your rule into all the states you want it to be matched in. For example, to tokenized JS-style string interpolation such as a\${c: d}e, you might use:
Errors If none of your rules match, Moo will throw an Error, since it doesn't know what else to do. If you prefer, you can have moo return an error token instead of throwing an exception. The error token will contain the whole of the rest of the buffer. <pre>moo.compile({ // ... myError: moo.error,</pre>	<pre>// <= `a\${c: d}`e` // => `strat`const interp `lbrace ident colon space ident `lbrace   `lbrace const `strand  The `lbrace` rule is annotated with pop, so it moves from the main state into either lit or main, depending on the stack.</pre>	<pre>lit: { interp: {match: '\$\$', push: 'main'}, escape: /\\"/, `strand: {match: '`', pop: 1}, const: {match: /(?:[\$'"] \\\$ `)?\{\}\)+/, lineBreaks: true}, }, })</pre>	<pre>let lexer = moo.states({ main: { `stratrt: {match: '`', push: 'lit'}, ident: /\w+/, lbrace: {match: '{', push: 'main'}, rbrace: {match: '}', pop: 1}, colon: ':', space: {match: /\s+/, lineBreaks: true}, }</pre>

<pre> }) moo.reset('invalid') moo.next() // -> { type: 'myError', value: 'invalid', text: 'invalid', offset: 0, lineBreaks: 0, line: 1, col: 1 } moo.next() // -> undefined </pre> You can have a token type that both matches tokens <i>and</i> contains error values. 33	<pre> moo.compile({ // ... myError: {match: /[\\\$?`]/, error: true}, }) </pre> Formatting errors If you want to throw an error from your parser, you might find <code>formatError</code> helpful. Call it with the offending token: 34	<pre> throw new Error(lexer.formatError(token, "invalid syntax")) </pre> It returns a string with a pretty error message. Error: invalid syntax at line 2 col 15: <pre> totally valid `syntax` ^ </pre> 35	Iteration Iterators: we got ‘em. <pre> for (let here of lexer) { // here = { type: 'number', value: '123', ... } } </pre> Create an array of tokens. <pre> let tokens = Array.from(lexer); </pre> 36
40	39 Do check the FAQ. Before submitting an issue, remember... Contributing <pre> }) // ...], value: x => x.slice(1, -1)}, \])*?"/, lineBreaks: true, {match: /(?:\\(?:[\\n] ["'\\]))?/ </pre> 42	38 <pre> moo.compile({ STRING: [{match: /""[^\s"]*/}, {match: /' '[^\s']*'/}, {match: /(?:\\(?:[\\n] ["'\\]))?/}, {match: /(?:\\(?:[\\n] ["'\\]))?/}, {match: /(?:\\(?:[\\n] ["'\\]))?/}, value: x => x.slice(1, -1)},], }) </pre> value(), which will be called on the value before storing it in the Token object. 43	37 Transform Use it's iteration tools with Moo. <pre> for (let [here, next] of itt(lexer).lookahead()) { // pass a number if you need more tokens // enjoy! } </pre> Moo doesn't allow capturing groups, but you can supply a transform function, 44
41	42	43	44
48	47	46	45